

Laboratory 6

TASK: MULTITHREADED APPLICATION USING MUTEX LOCKS (C++)

This laboratory focuses on using **Mutex Locks** and **shared variables** to prevent race conditions in a multithreaded ticket-selling simulation. Students will practice thread synchronization and safe access to shared resources in **C++ on Linux/macOS**.

Scenario

- There are **250 tickets** available.
- The system uses **4 to 10 threads**, depending on hardware.
- Each thread sells tickets until none remain.
- Prevent overselling tickets and deadlocks
- Simulate selling a ticket transaction using `sleep_for()` function.

Tasks:

1. Create a C++ multithreaded program that sells tickets.
 - Use `std::thread` to spawn 4–10 worker threads.
 - Use a **shared integer** representing the number of tickets. (1 point)
2. Synchronize all ticket-selling operations using a Mutex Lock.
 - Use mutex to protect the critical section (ticket decrement).
 - Explicitly call `lock()` and `unlock()`, or use `std::lock_guard`.
 - Wait for all threads to complete.
 - Prevent race conditions and deadlock. Ensure no overselling; the ticket counter never becomes negative. (1.5 points)
3. Pass multiple arguments to each thread using a struct.
 - Include: Thread ID, Movie title, current time and date.
 - Each thread prints its ID, movie title, current time and date. (1 point)
4. Simulate a transaction delay.
 - Use `sleep_for()` inside the critical section or after it. (0.5 point)
5. Add comments to your code (explain used functions and sections). (0.5 point)
6. Measure execution time using a timing function (0.5 point)

NOTE: Upload the print-screen of final code execution on enauczanie and of course your C++ code :)